



Sistemas Distribuidos I (75.74)

Sistemas elásticos y de alta disponibilidad Arquitecturas orientadas a servicio

Técnicas de Diseño. Elasticidad. Disponibilidad.

Docentes

- Pablo D. Roca
- Guido Albarello
- Gabriel Robles
- Franco Barreneche
- Tomás Nocetti



- **Escalabilidad**
- Elasticidad
- Alta Disponibilidad
- Arquitecturas Orientadas a Servicio



Escalabilidad | Objetivos

El objetivo de escalabilidad es el crecimiento,

- ***Respecto del tamaño:*** Agregando usuarios o recursos a controlar.
- ***Respecto de la distribución geográfica:*** Permitiendo dispersión.
- ***Respecto de los objetivos administrativos del sistema:*** Nuevas sintaxis, semánticas y servicios ofrecidos.



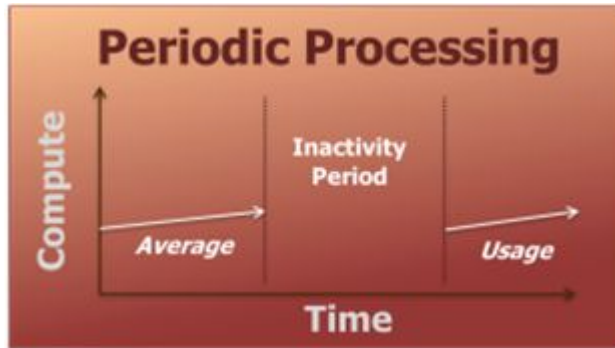
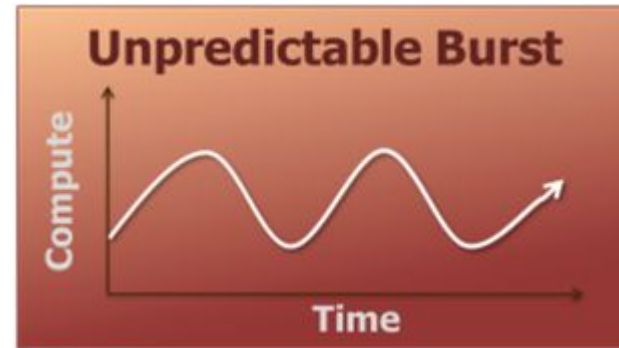
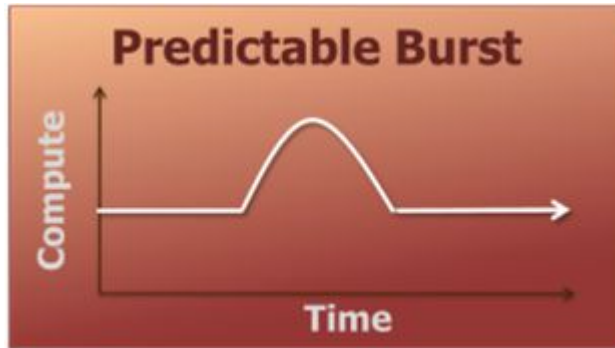


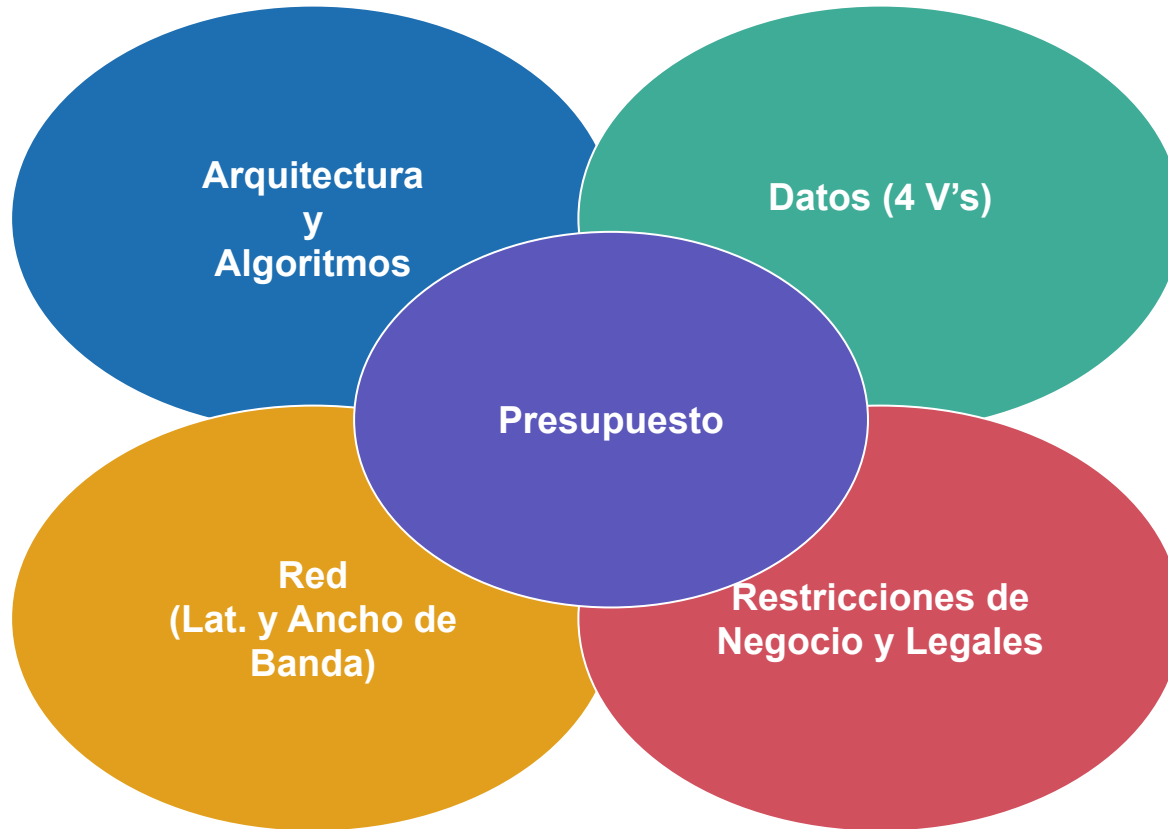
Escalabilidad | Características de las Plataformas

- **Plataformas para alta concurrencia**
 - Aplican patrones ya conocidos y probados
 - Escalamiento automático (con ciertos límites)
 - Vinculación fuerte con una infraestructura o producto
- **Arquitecturas Ad-Hoc y Personalizadas**
 - Necesidad de configuración y soporte
 - Escalamiento manual o automatizado por humanos
 - Posibilidad de migraciones a distintas plataformas



Escalabilidad | Patrón de Carga de Aplicaciones Web







- **Escalamiento vertical**
 - Agregar recursos a un nodo
- **Escalamiento horizontal**
 - Redundancia
 - Balanceadores de carga
 - Proximidad geográfica
- **Fragmentación de datos**
 - Fraccionar para optimizar.
 - Mantener juntos datos "ceranos"
- **Componentización**
 - Separar servicios
- **Optimizar algoritmos**
 - Performance
 - Mensajería
- **Asincronismo**
 - Mantener sincrónico sólo lo estrictamente necesario
 - Limitado por el negocio

Agenda



- ☐ Escalabilidad
- ☒ **Elasticidad**
- ☐ Alta Disponibilidad
- ☐ Arquitecturas Orientadas a Servicio



- **Escalabilidad**

- Capacidad de un sistema para poder adaptarse a diferentes ambientes (*environments*) modificando los recursos del sistema
- Término utilizado en muchos contextos

- **Elasticidad**

- Capacidad de un sistema para poder modificar dinámicamente los recursos del sistema adaptándose a patrones de carga
- Término utilizado en Arquitecturas Cloud. Requiere soporte de la infraestructura



Elasticidad | Componentes

- ***Application Load Balancer***

- Servicios / instancias nuevas reciben tráfico
- Servicios / instancias dados de baja / caídos / degradados dejan de recibir tráfico
- Cómo detectamos estado de un servicio / instancia ?

- ***Autoscaler***

- *Scale In/Scale Out* en función de las métricas recolectadas
- *Scale Out*: Incrementa instancias
- *Scale In*: Decrementar instancias

- **Monitoring Automático**

- Métricas sobre CPU, memoria, I/O, *networking*, etc. por cada servicio / instancia

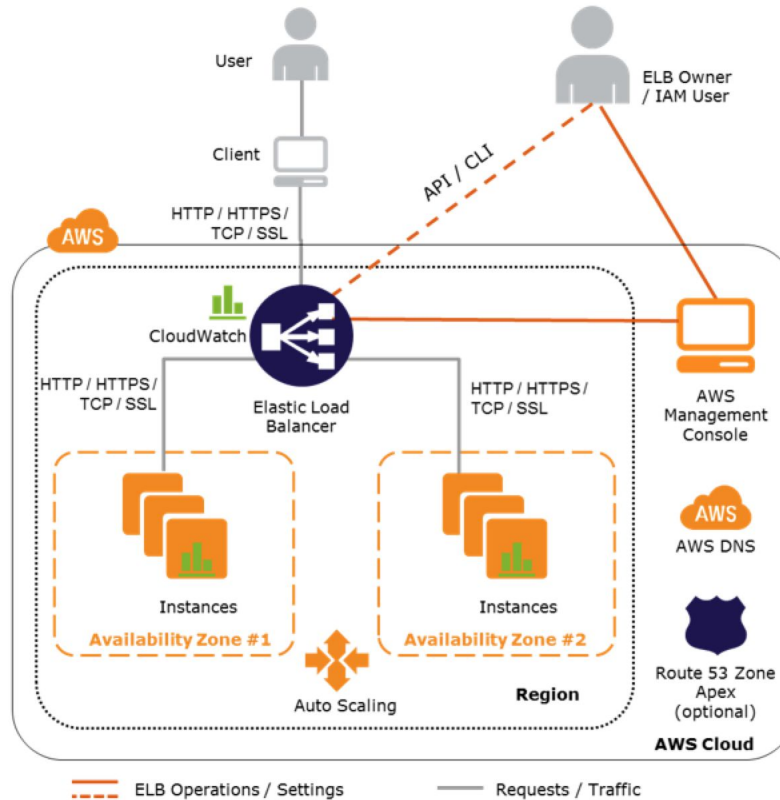


Elasticidad | Ejemplo N#1 - AWS

- **Application Load Balancer** ([Amazon Elastic Load Balancer](#))
 - Redirecciona tráfico a instancias (EC2), incluso en diferentes zonas
 - Interactúa con instancias para verificar estado (*/status* endpoint)
- **Autoscaler** ([Amazon Autoscaling](#))
 - Autoscaling group permite definir *min*, *desired* and *max* instances
 - Diferentes policies para lograr diferentes objetivos (dynamic vs manual scaling)
- **Monitoring Automático** ([Amazon CloudWatch](#))
 - Métricas globales automatizadas
 - Admite [agregar métricas propias](#)



Elasticidad | Ejemplo N#1 - AWS





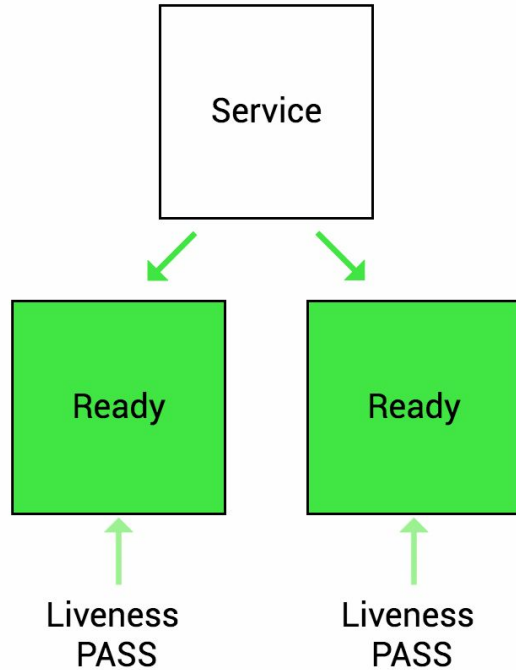
Elasticidad | Ejemplo N#2 - Kubernetes

- **Application Load Balancer** ([Kubernetes Service](#))
 - Redirecciona tráfico a pods en estado **Ready**
 - [Liveness and Readiness probes](#) permiten al usuario indicar si un pod debe recibir tráfico o no
- **Autoscaler** ([Horizontal Pod Autoscale](#))
 - [Se crea un HPA resource](#) asociado a un deployment en función de alguna métrica
 - Similar a como funciona el autoscaler de AWS, se pondera las métricas de cada instancia/container
 - $desiredReplicas = ceil[currReplicas * (currMetricValue / desiredMetricValue)]$
- **Monitoring Automático** ([Kubernetes Metrics Server](#))
 - Collecta métricas de cada pod en el cluster y las expone en el API server
 - Diseñado específicamente para autoscaling

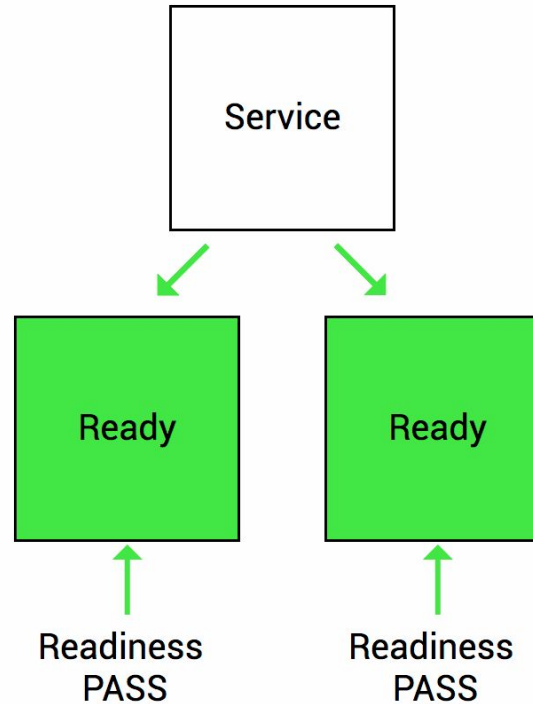


Elasticidad | Ejemplo N#2 - Kubernetes

Liveness



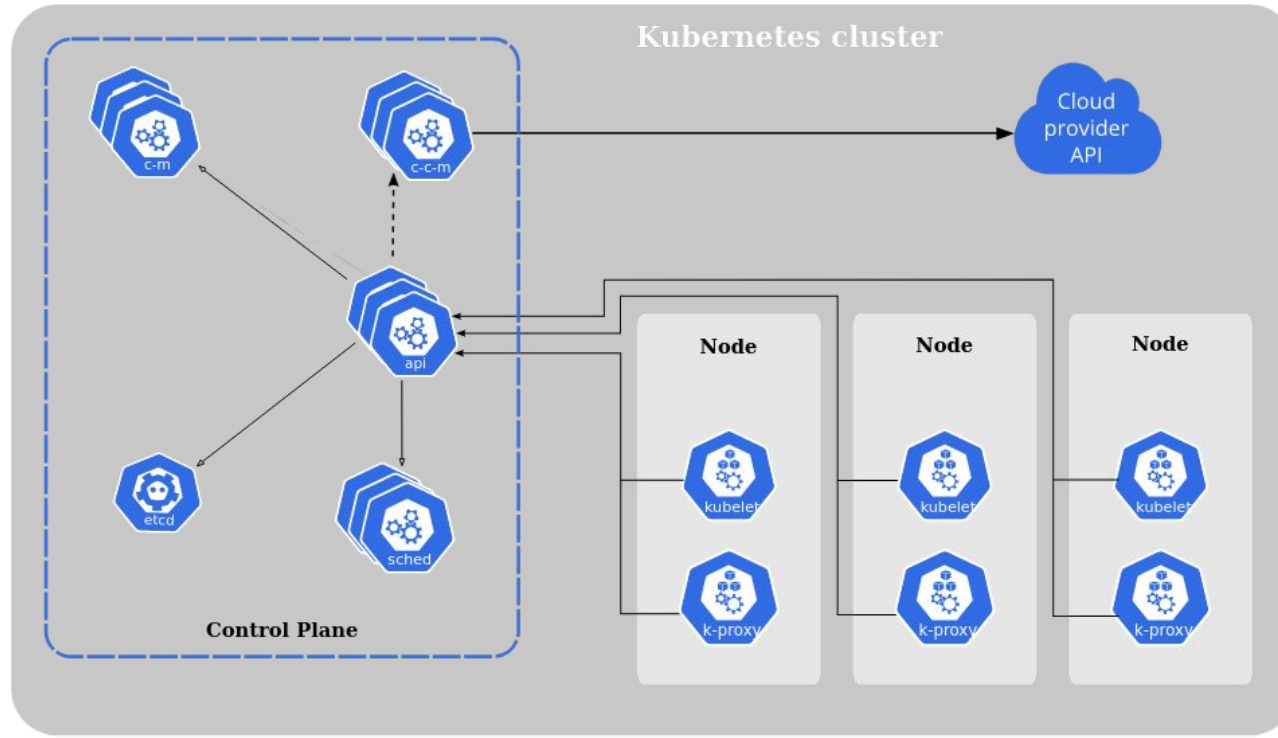
Readiness



<https://cloud.google.com/blog/products/containers-kubernetes/kubernetes-best-practices-setting-up-health-checks-with-readiness-and-liveness-probes>



Elasticidad | Ejemplo N#2 - Kubernetes



Agenda



- ☐ Escalabilidad
- ☐ Elasticidad
- ☒ **Alta Disponibilidad**
- ☐ Arquitecturas Orientadas a Servicio



Alta Disponibilidad | Introducción

Aún cuando ciertas propiedades de confiabilidad fallen para sistemas públicos, nos interesa que siempre estén disponibles:

$$P(\textit{system available}) = 1 - P(\textit{failure})$$

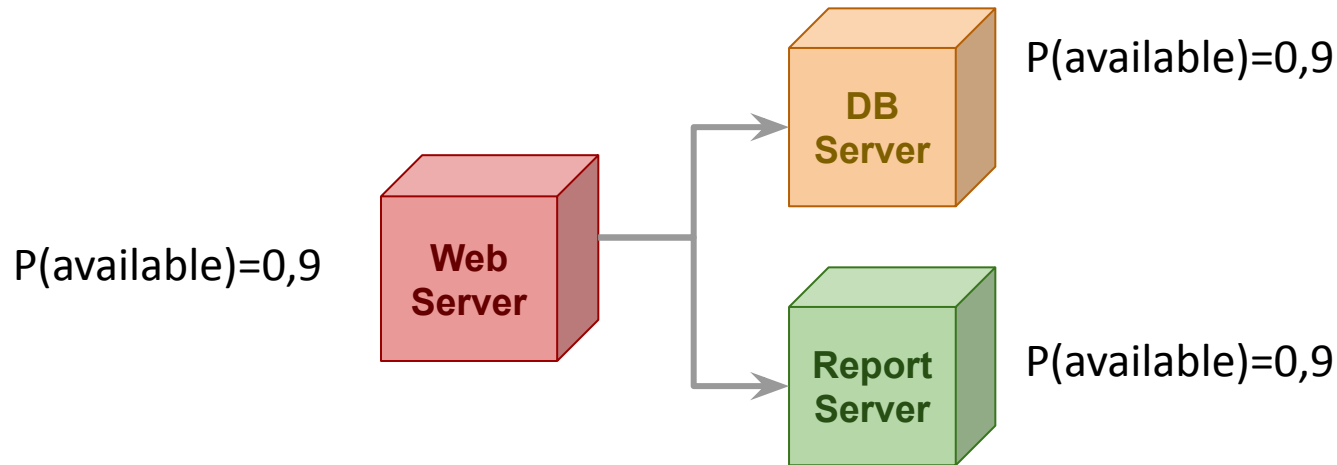
Como $P(\textit{system available}) < 1$ siempre, lo importante es definir cuán cerca de 1 se encuentra:

- $P(\textit{system available}) \sim 0,9 \Rightarrow 36,5$ días caídos al año
- $P(\textit{system available}) \sim 0,999 \Rightarrow 8,76$ horas caídas al año

La disponibilidad de un sistema se mide por la cantidad de 9's



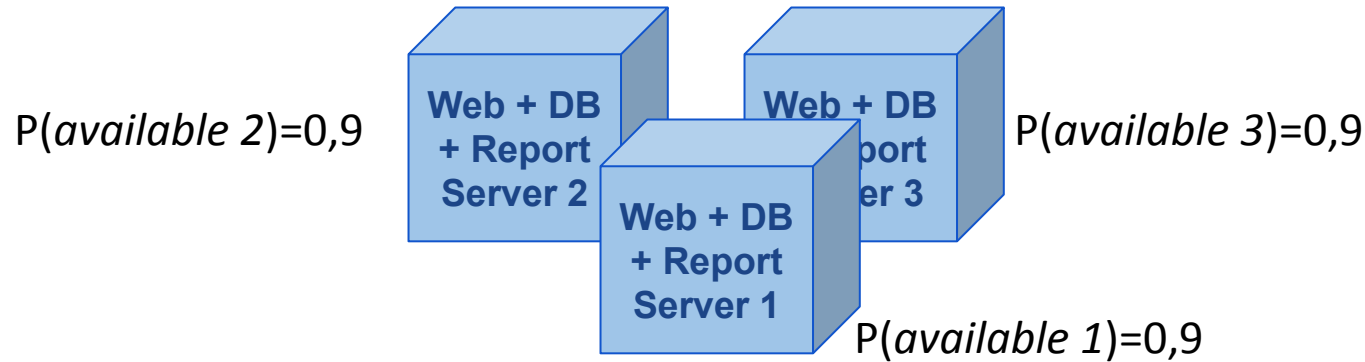
Escenario: Sistema distribuido con un componente por nodo



$$\begin{aligned} P(\text{available}) &= P(\text{webAvailable}) \cdot P(\text{dbAvailable}) \cdot P(\text{reportAvailable}) \\ &= 0,9 \cdot 0,9 \cdot 0,9 = \mathbf{0,729} \end{aligned}$$



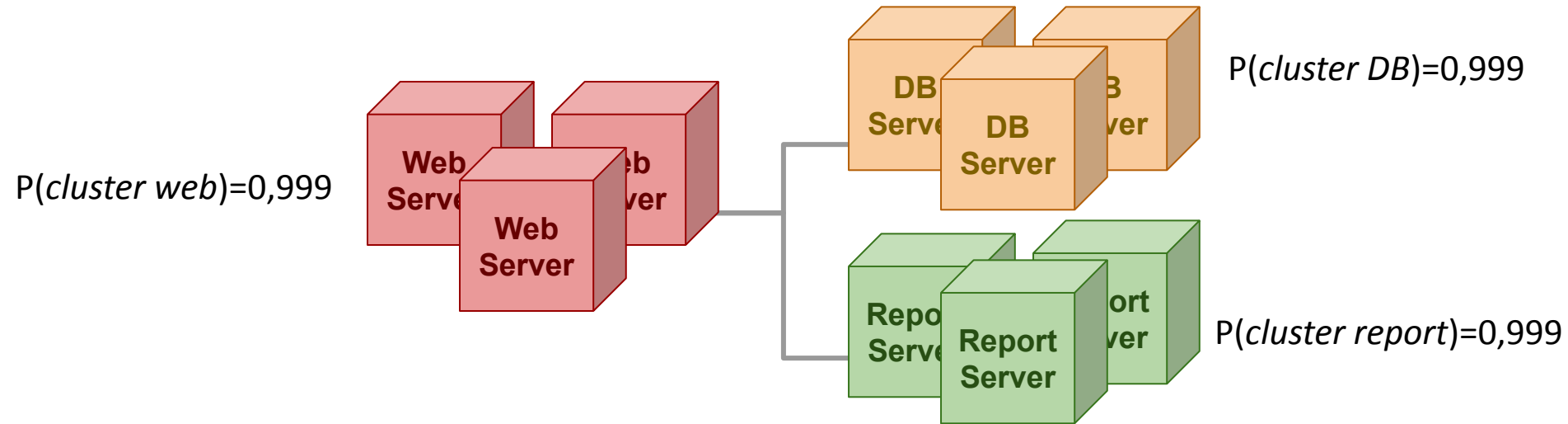
Escenario: Sistema con redundancia con todos los componentes en c/nodo



$$\begin{aligned} P(\text{available}) &= 1 - P(\text{failure}) = 1 - P(\text{failure } 1 \ \& \ \text{failure } 2 \ \& \ \text{failure } 3) \\ &= 1 - 0,1 \cdot 0,1 \cdot 0,1 = 1 - 0,001 = \mathbf{0,999} \end{aligned}$$



Escenario: Sistema distribuido con clusters para cada componente



$$\begin{aligned} P(\text{available}) &= P(\text{cluster web}) \cdot P(\text{cluster DB}) \cdot P(\text{cluster report}) \\ &= 0,999 \cdot 0,999 \cdot 0,999 = \mathbf{0,997} \end{aligned}$$



Alta Disponibilidad | Terminología en la industria

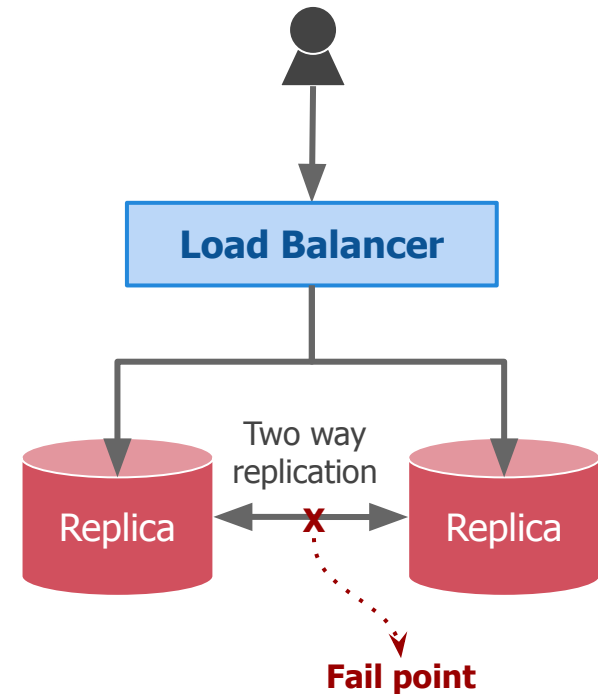
- **SLA: Service Level Agreement**
 - Contrato / Acuerdo de disponibilidad pactado con el cliente
 - Definición de qué sucede si el mismo no se respeta (e.g. [BigQuery SLA](#))
- **SLO: Service Level Objectives**
 - Lo que se debe cumplir para no invalidar el SLA
 - E.g. Availability > 99.95%
- **SLI: Service Level Indicators**
 - Métricas a ser comparadas con los SLOs
 - Siempre deben ser superiores al threshold del SLO
 - Por lo general requiere una plataforma de [observability](#)
 - Analizar impacto del [despliegue de los servicios](#)



CAP Theorem | Problemas frente a particionamiento

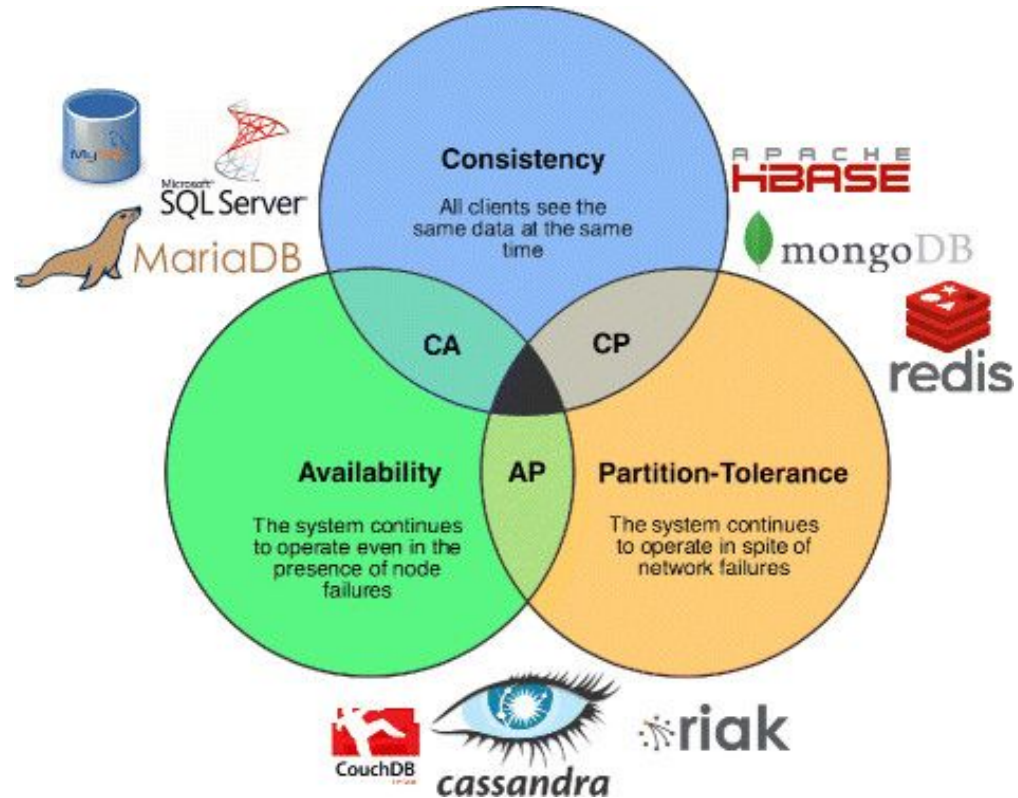
Eric Brewer ('98): en sistemas distribuidos con almacenamiento es posible garantizar sólo 2 de los siguientes atributos:

- **Consistency:** también conocido como repetibilidad de respuesta de todos los nodos frente a un mismo pedido.
- **Availability:** capacidad del sistema de responder a todo pedido.
- **Partition Tolerance:** capacidad de lidiar con la formación grupos aislados de nodos.





- Se puede sacrificar *Consistency* o *Availability*. Pero no necesariamente es todo o nada.
- Sacrificar *Partition Tolerance* significa no proveer un sistema distribuido.

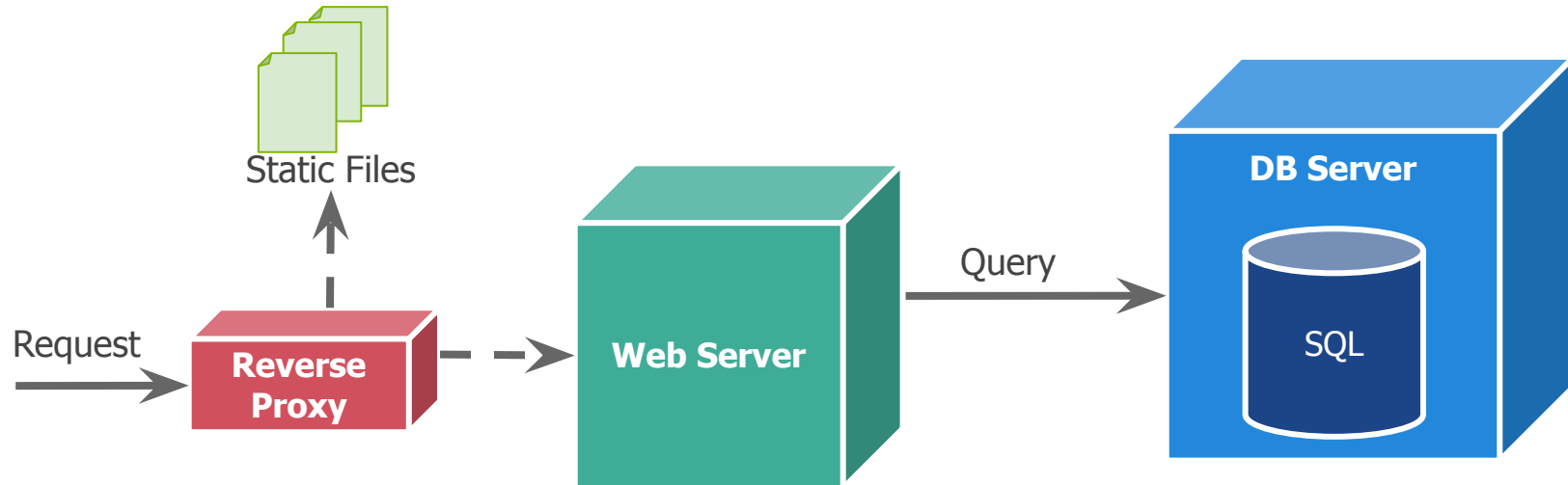


Agenda



- Escalabilidad
- Elasticidad
- Alta Disponibilidad
- **Arquitecturas Orientadas a Servicio**

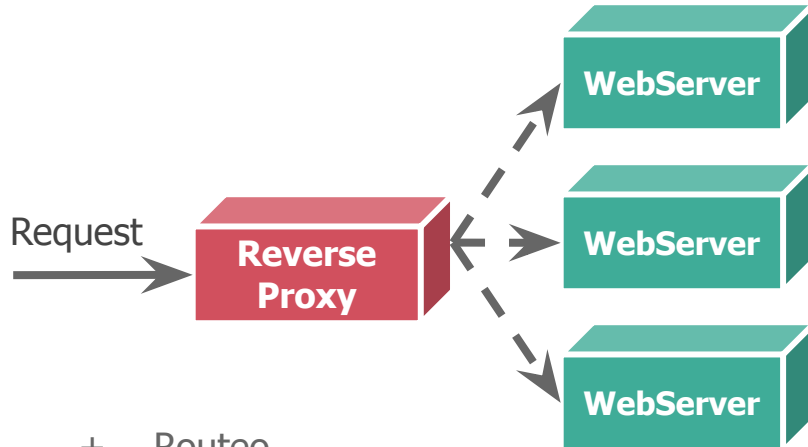
Evolución en Arquitecturas | Monolíticas





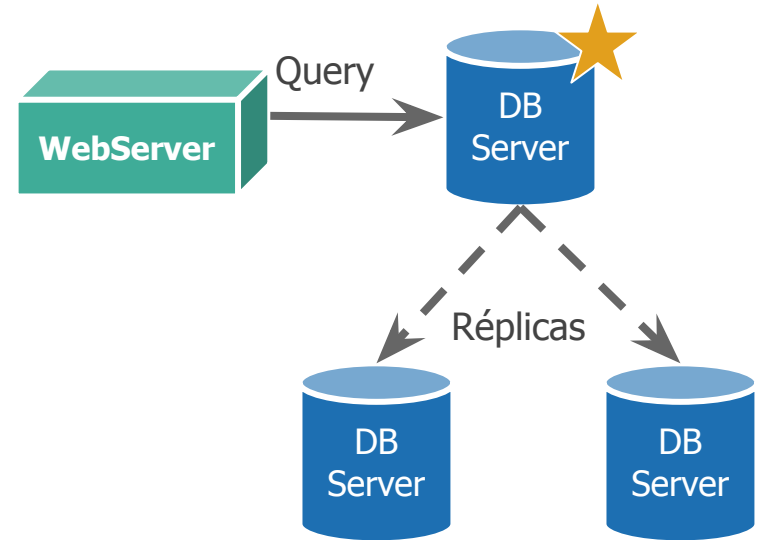
Evolución en Arquitecturas | Monolíticas (Escalables)

Vista de *Web Requests*



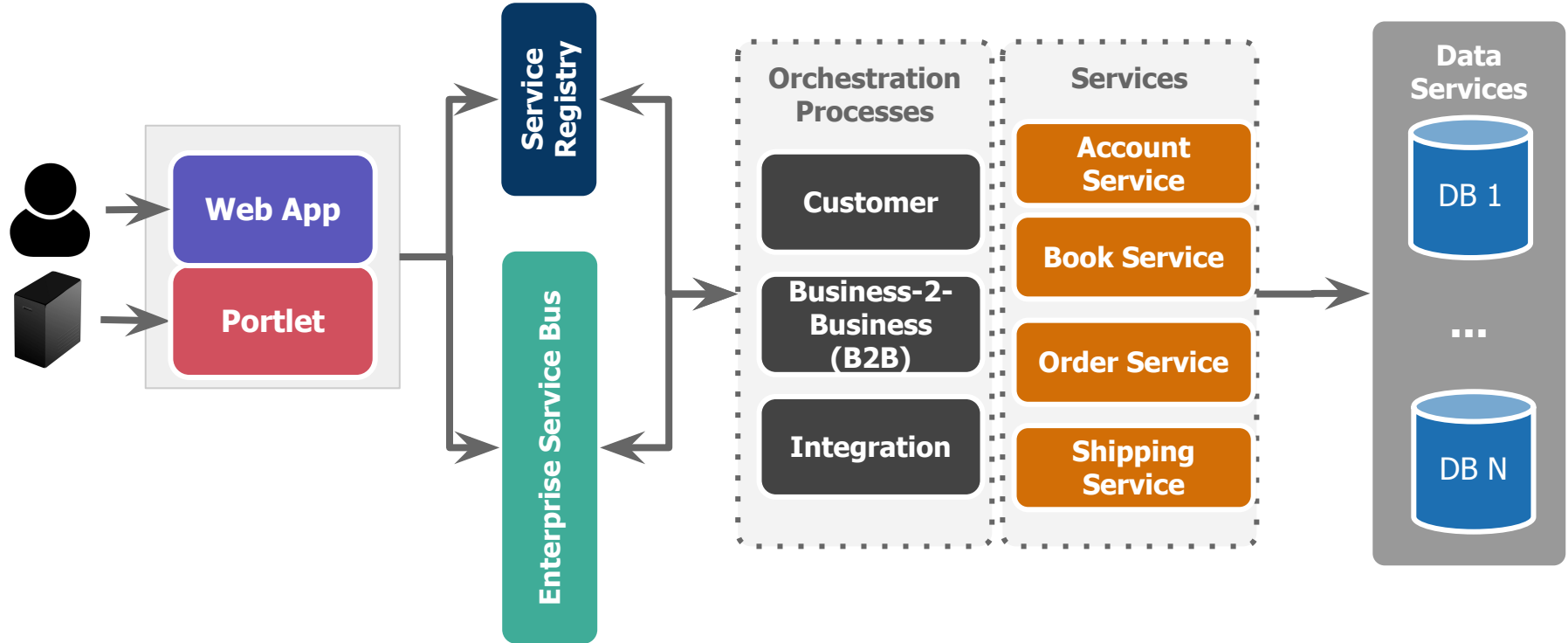
- + Routeo
- + Escalabilidad
- *Single point of failure*

Vista de *Data Queries*



- + *Throughput* en Lectura
- *Throughput* en Escritura

Evolución en Arquitecturas | *Service Oriented Arch. (SOA)*





- SOA no es únicamente la definición arquitecturas sino un paradigma orientado al ámbito corporativo.
- Se complementa con el concepto de Business process management (BPM):

"... discipline involving any combination of modeling, automation, execution, control, measurement and optimization of business activity flows, in support of enterprise goals, spanning systems, employees, customers and partners within and beyond the enterprise boundaries"

Palmer, Nathaniel. "What Is BPM". bpm.com. Retrieved 20 January 2018.

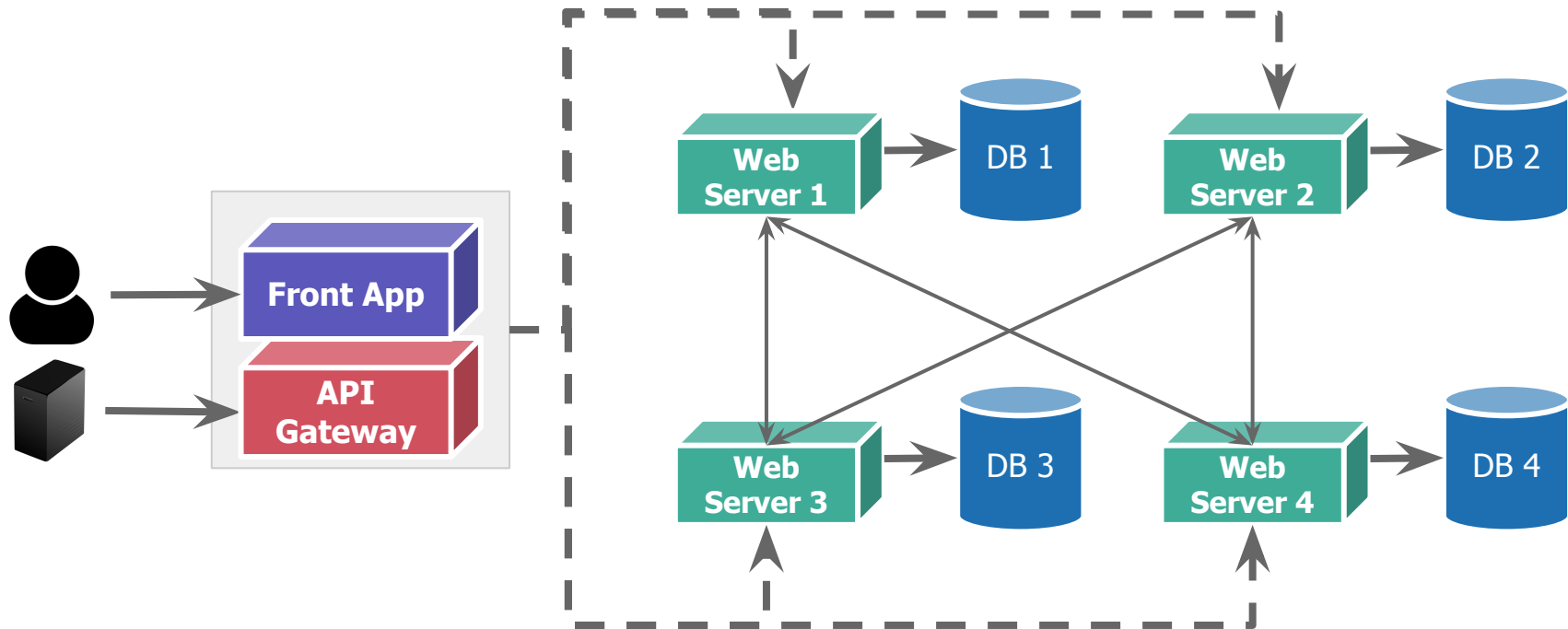


Tecnologías

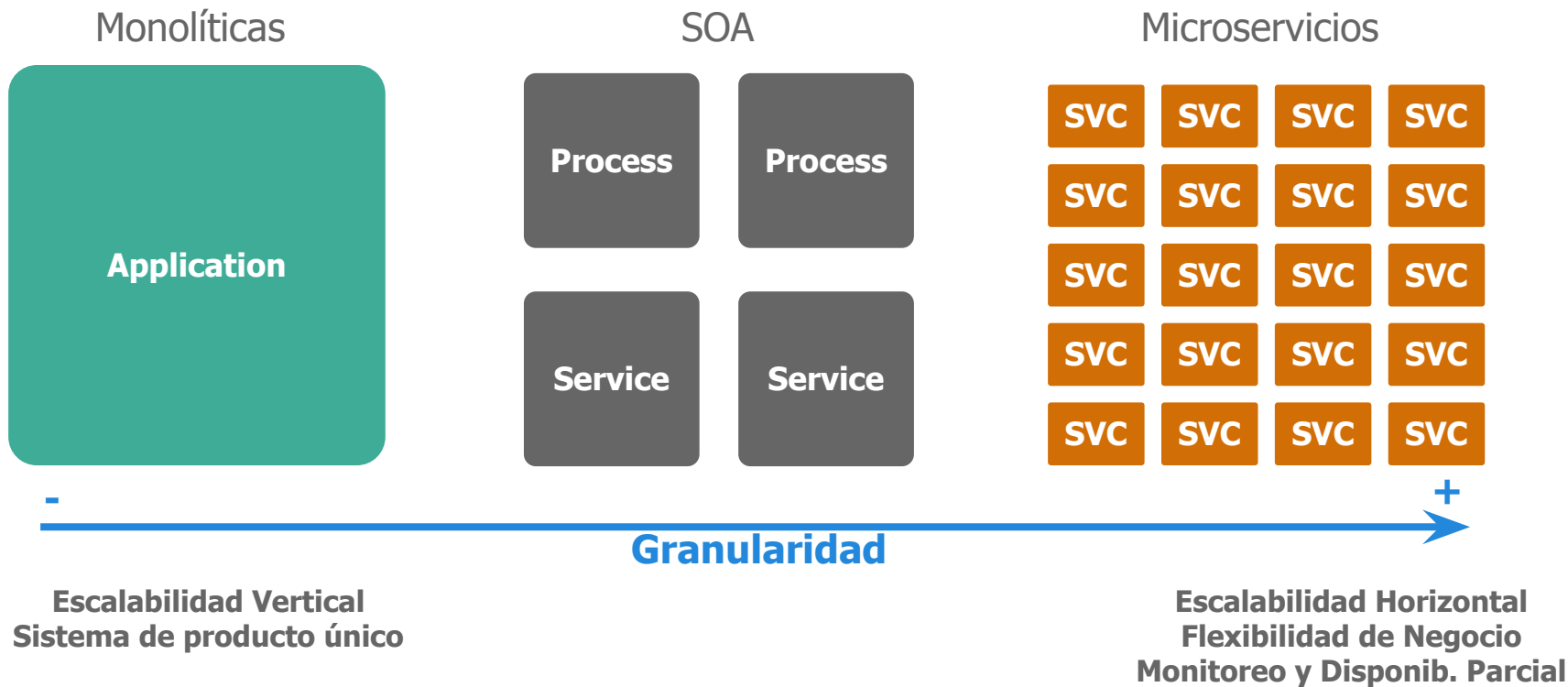
- WebServices = SOAP + HTTP
- ESB preponderante para eventos
- *Service Repository & Discovery* para comunicación punto a punto

Procesos y Servicios

- Contract
- Interface
- Implementation: Business Logic + Data Management

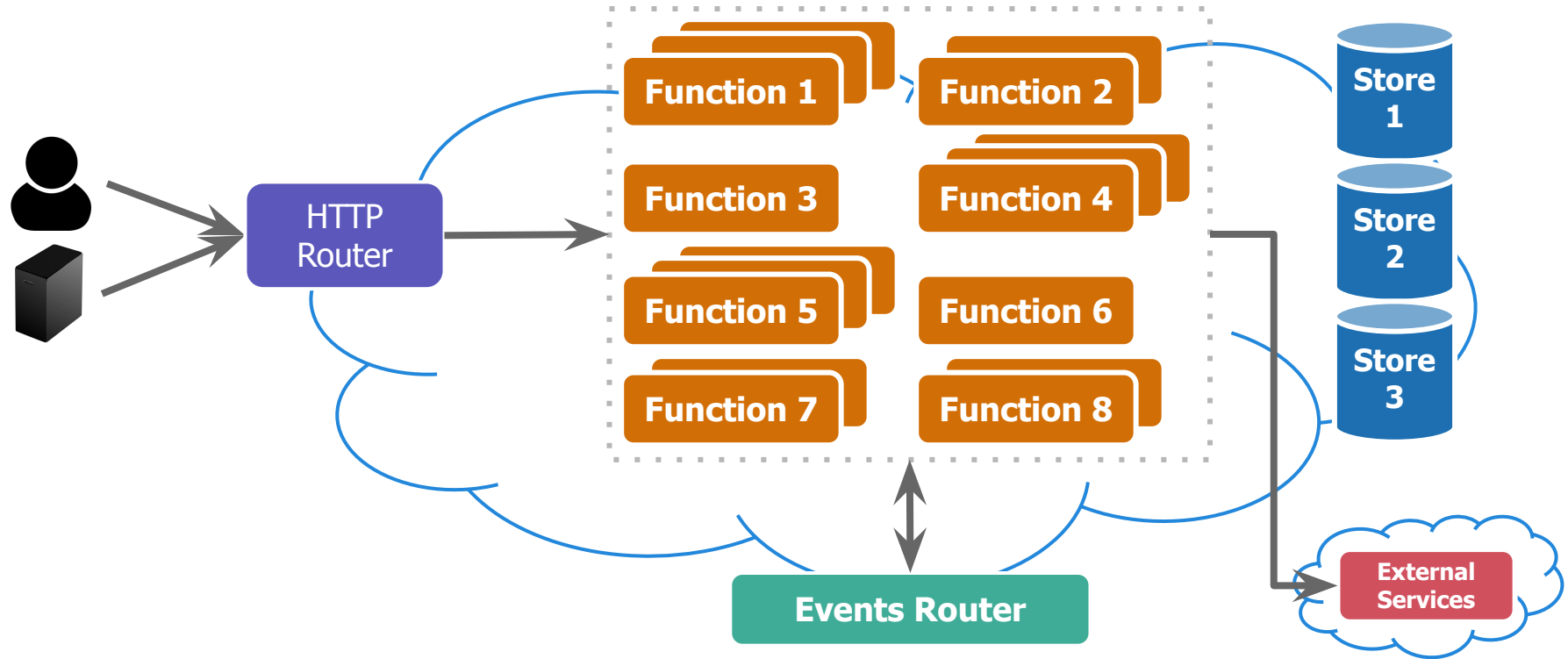


Evolución en Arquitecturas | Transición entre Arquitecturas





Evolución en Arquitecturas | *Serverless*







Bibliografía

- G. Coulouris, J. Dollimore, t. Kindberg, G. Blair: Distributed Systems. Concepts and Design, 5th Edition, Addison Wesley, 2012.
 - Capítulo 18: Coordination and Agreement
- M. Van Steen, A. Tanenbaum: Distributed Systems. 3rd Edition. Pearson Education, 2017.
 - Capítulo 1.2: Introduction - Design Goals
 - Capítulo 8.1: Fault Tolerance - Introduction to Fault Tolerance